

Week 3

General Purpose Input/Output (GPIO)

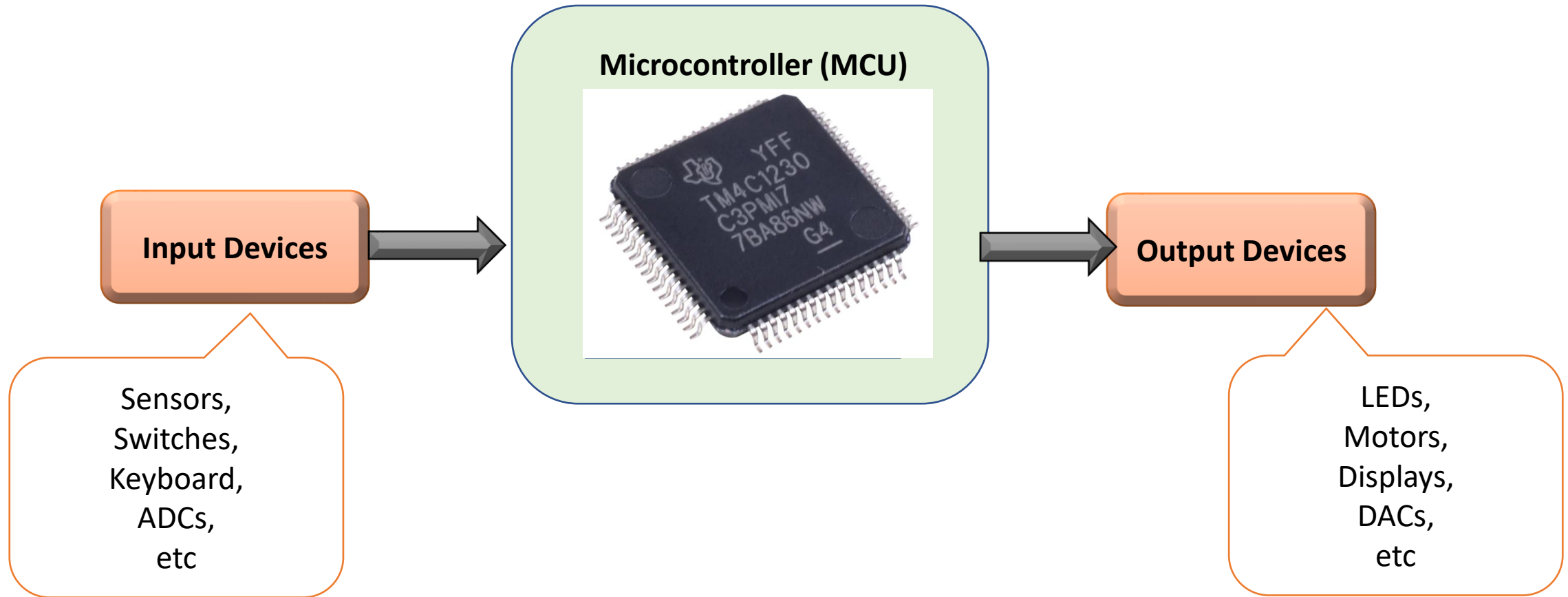
Topics

- Texas Instruments Tiva TM4C123GH6PM Microcontroller
 - GPIO
- I/O Pins in TI Tiva LaunchPad
- Direction and Data Registers
- Data Register (GPIODATA) in TI ARM
- Seven-segment LED interfacing and programming

Building Blocks of a Typical Microcontroller

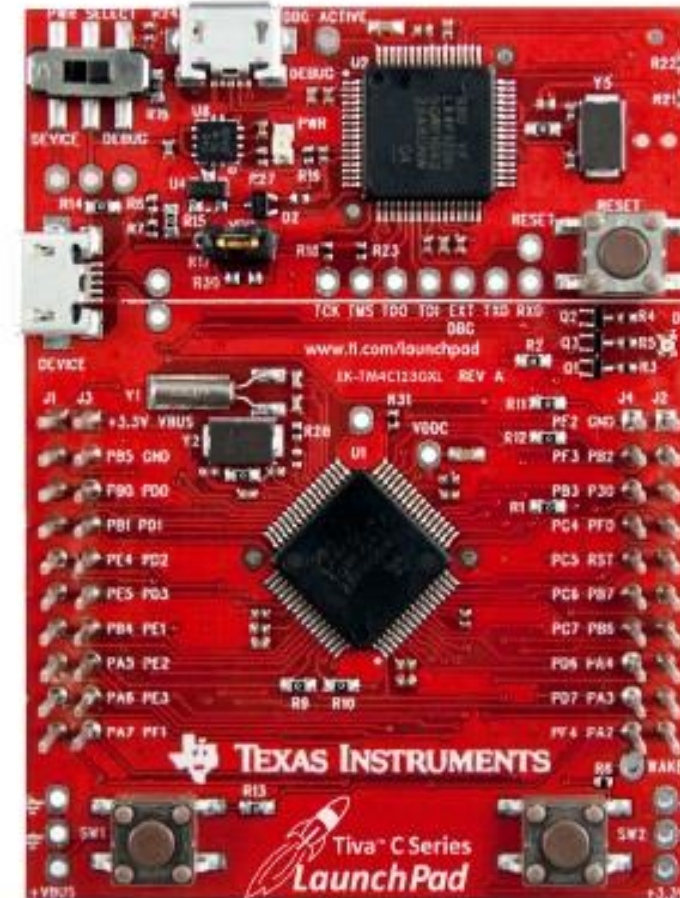


Building Blocks of a Typical Microcontroller



Tiva™ EK-TM4C123GXL LaunchPad

- ◆ ARM® Cortex™-M4F
64-pin 80MHz TM4C123GH6PM
- ◆ On-board USB ICDI
(In-Circuit Debug Interface)
- ◆ Micro AB USB port
- ◆ Device/ICDI power switch
- ◆ BoosterPack XL pinout also supports
existing BoosterPacks
- ◆ 2 user pushbuttons
- ◆ Reset button
- ◆ 3 user LEDs (1 tri-color device)
- ◆ Current measurement test points
- ◆ 16MHz Main Oscillator crystal
- ◆ 32kHz Real Time Clock crystal
- ◆ 3.3V regulator
- ◆ Support for multiple IDEs:



mentor
embedded

ST
IAR
SYSTEMS

ARM
KEIL
An ARM Cortex

CODE
Composer
Studio

TM4C123GH6PM Datasheet

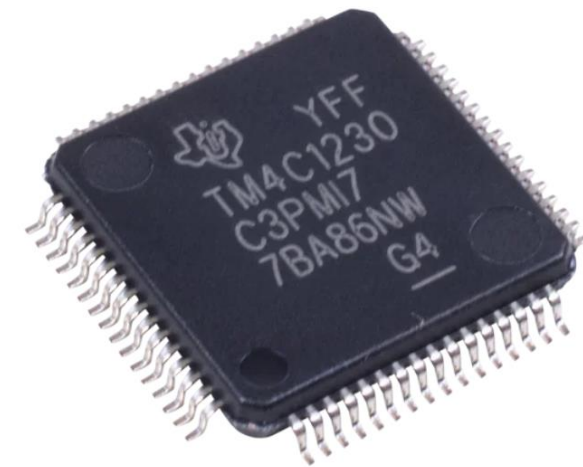
https://www.ti.com/lit/ds/symlink/tm4c123gh6pm.pdf?ts=1736926435485&ref_url=https%253A%252F%252Fwww.mouser.cn%252F

TM4C123GH6M Chip

TM4C123GH6M has 64 I/O pins.

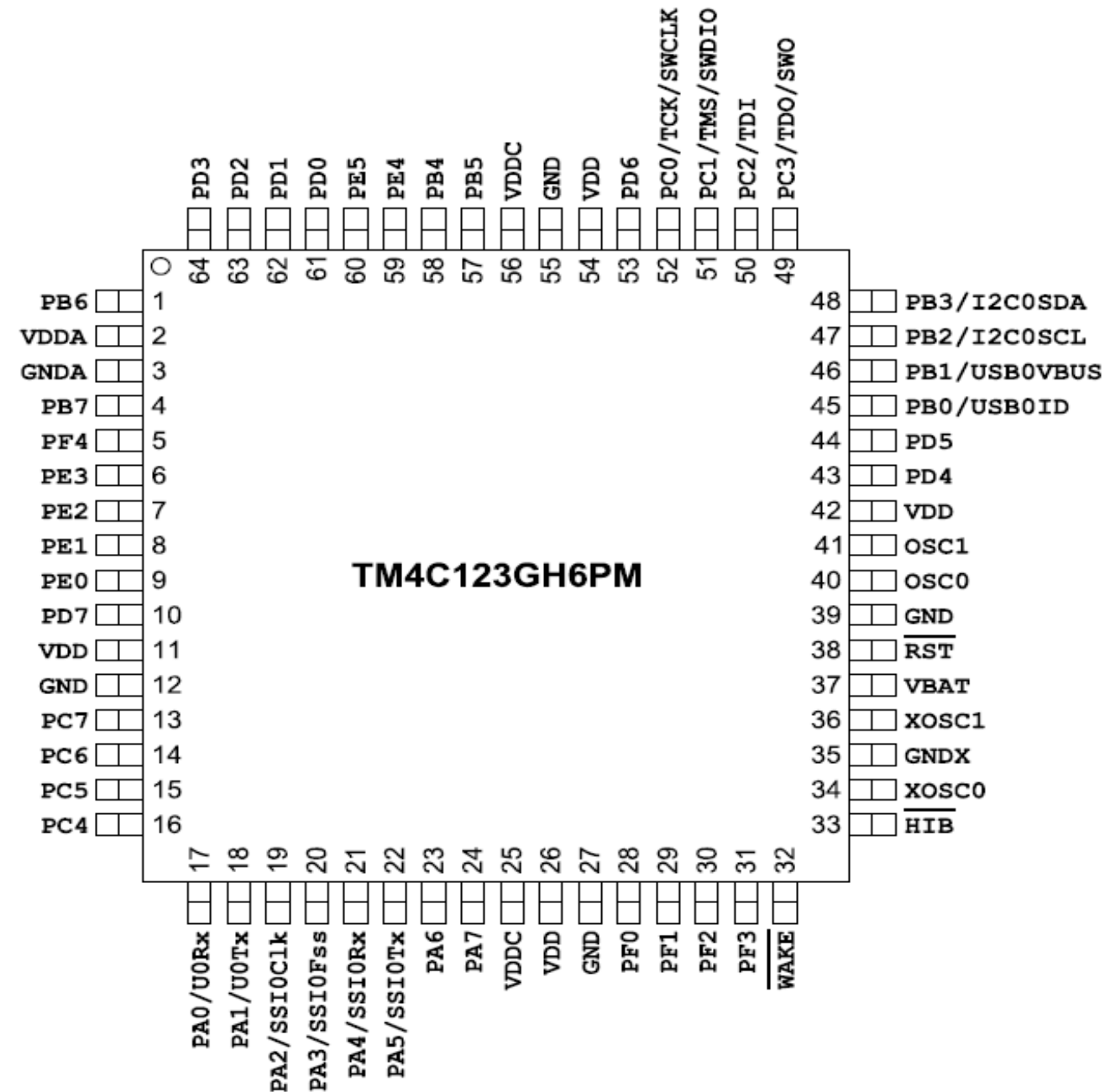
Up to 43 pins are used as GPIO pins that are arranged in 6 ports:

Port Name	Number of Pins
Port A	8 pins
Port B	8 pins
Port C	8 pins
Port D	8 pins
Port E	6 pins
Port F	5 pins



TM4C134GH6PM

Pin-out



Review

ARM is a **32-bit** microcontroller

What does it mean to have a 32-bit Bus Interface?

x-bit $\rightarrow 2^x$ combination

With 8-bit interface bus we have $2^8 = 256$. If we use each number as an address of a memory location, we can have 256 addressable memory locations.

With 32-bit interface bus, we have **$2^{32} = 4294967296 \sim 4\text{Gbytes}$** of addressable memory.

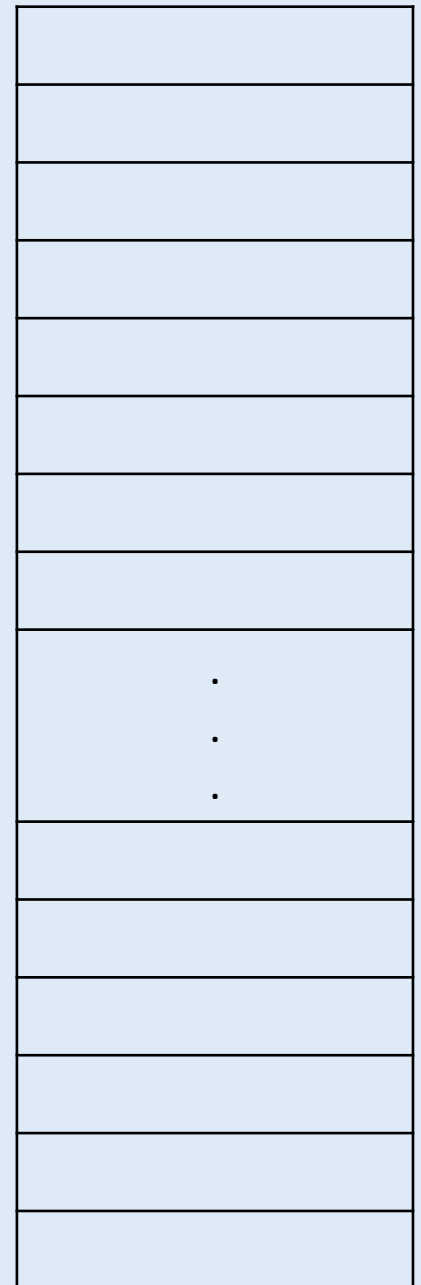
Memory Model

4294967295 = 0x FFFF,FFFF

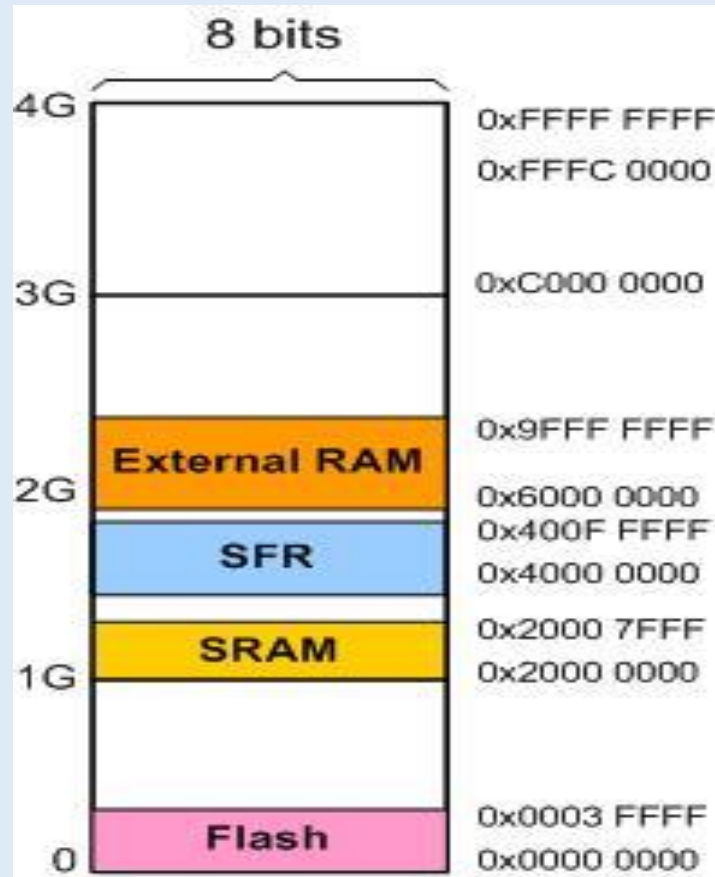
4GB of memory is available

1 = 0x 0000,0001

0 = 0x 0000,0000

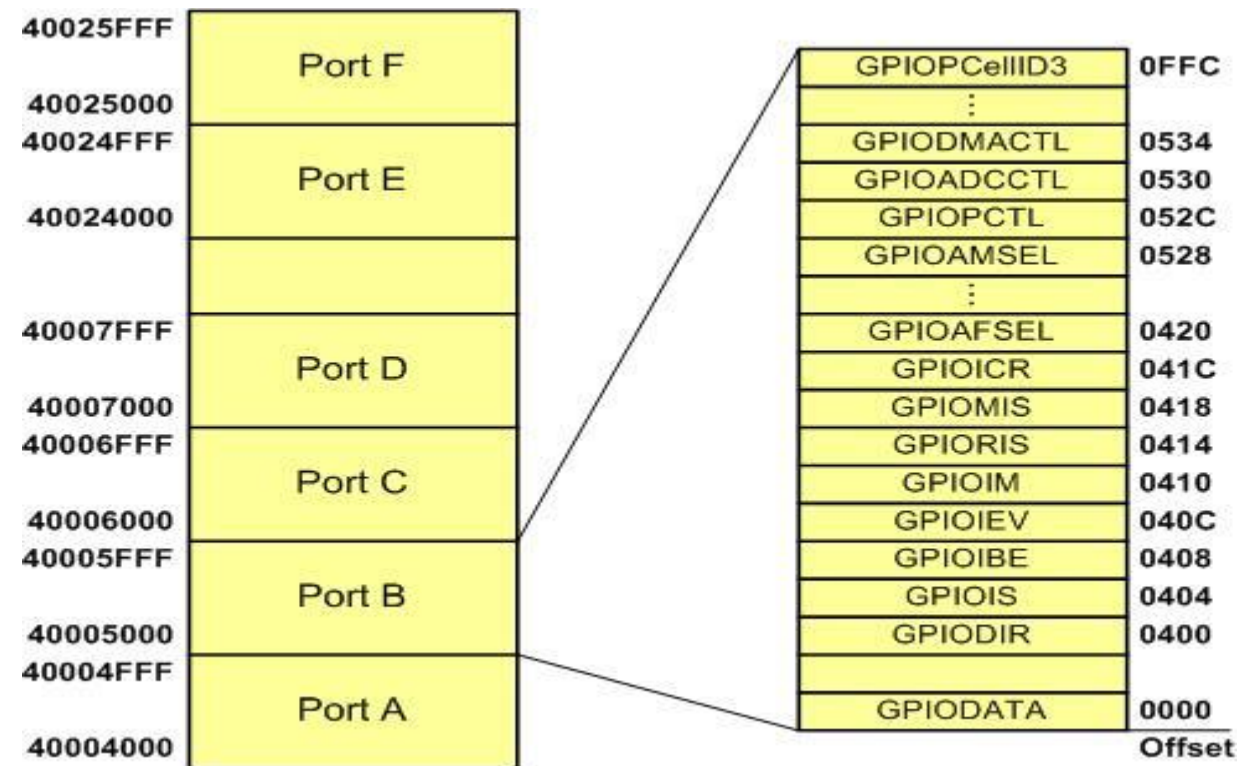


Memory Map



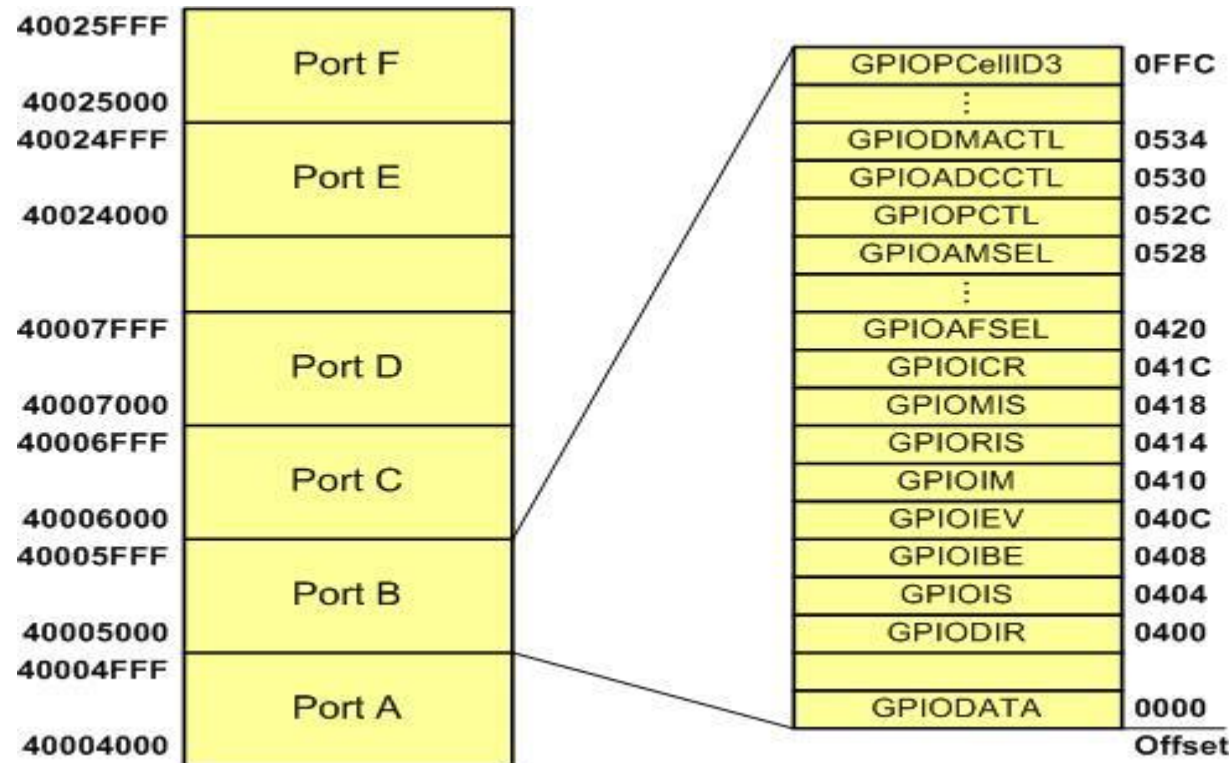
Type of Memory	Size	Address
Flash	256KB	0x0000,0000 to 0x0003,FFFF
SRAM	32KB	0x2000,0000 to 0x2000,7FFF
SFR (including GPIO)	All the peripherals	0x4000,0000 to 0x400F,FFFF

GPIO Memory Map



**4KB of
memory for
each port**

GPIO Memory Map



- Notice that total of 4KB of memory space is assigned to each of GPIO port. Why so many memory address locations?

See next slide for answer.

Why 4KB of memory address locations for each port?

- **Each GPIO port** requires a large number of **special function registers** to control and configure its behavior.
- These **special function registers** provide the **GPIO ports with a rich feature set**, allowing for advanced operations like:
 - Bit-level access and control.
 - Efficient I/O operations and configuration.

GPIO and Special I/O Modes

I/O ports have two operation modes:

Our Focus Today

General purpose I/O (GPIO):

- used for interfacing simple input and output devices with microcontroller.

Special purpose I/O mode:

- used for special functions such as timer, interrupt, analog to digital (ATD) converters, communication interfaces, etc.

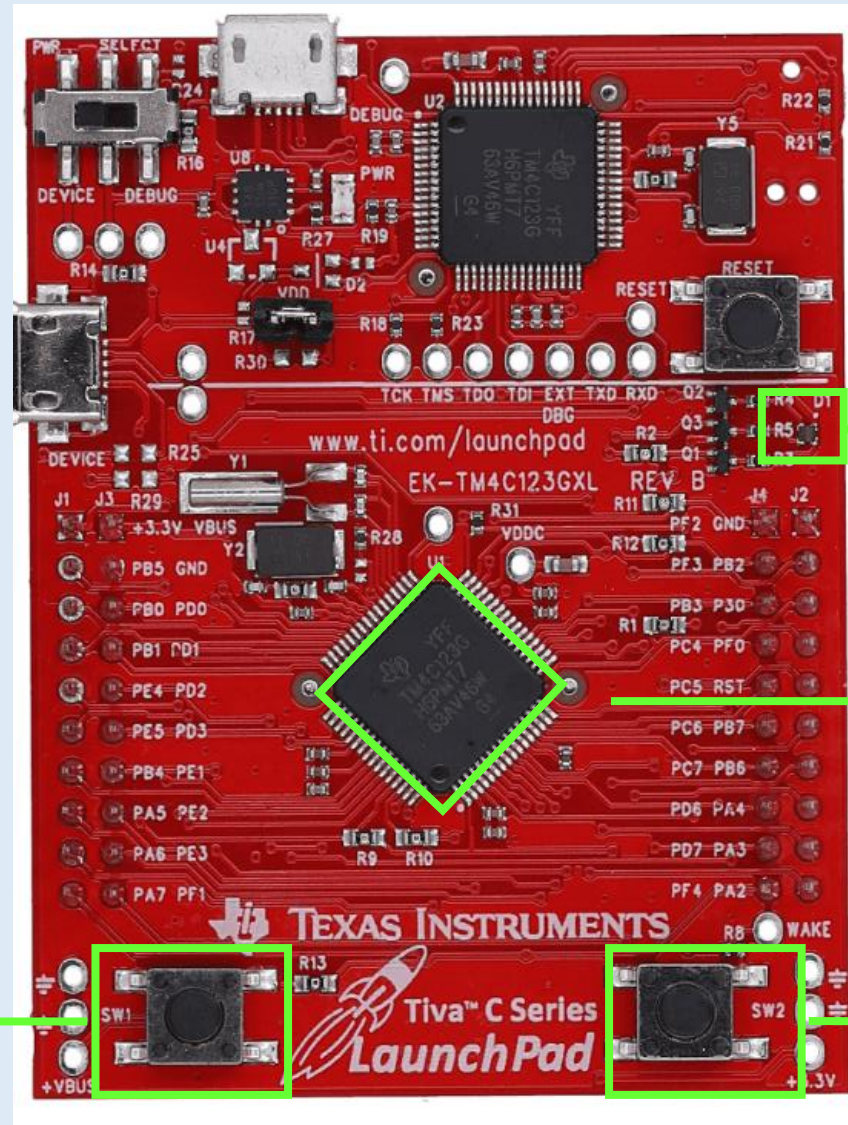
USB host

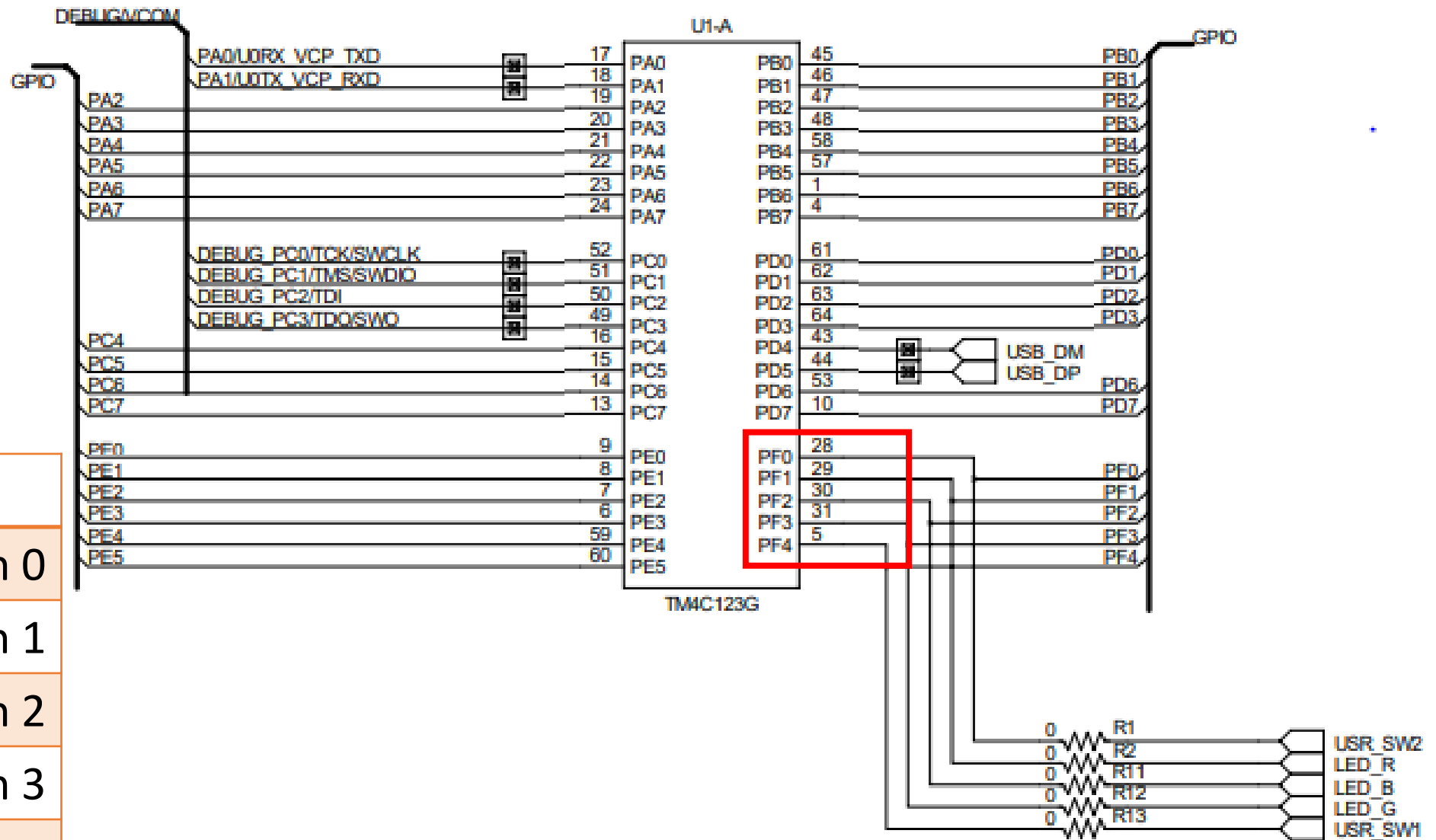
RGB LED

TM4C123GH6PM

Input Switch 1

Input Switch 2





Device	GPIO
Switch 2	GPIOF Pin 0
Red LED	GPIOF Pin 1
Blue LED	GPIOF Pin 2
Green LED	GPIOF Pin 3
Switch 1	GPIOF Pin 4

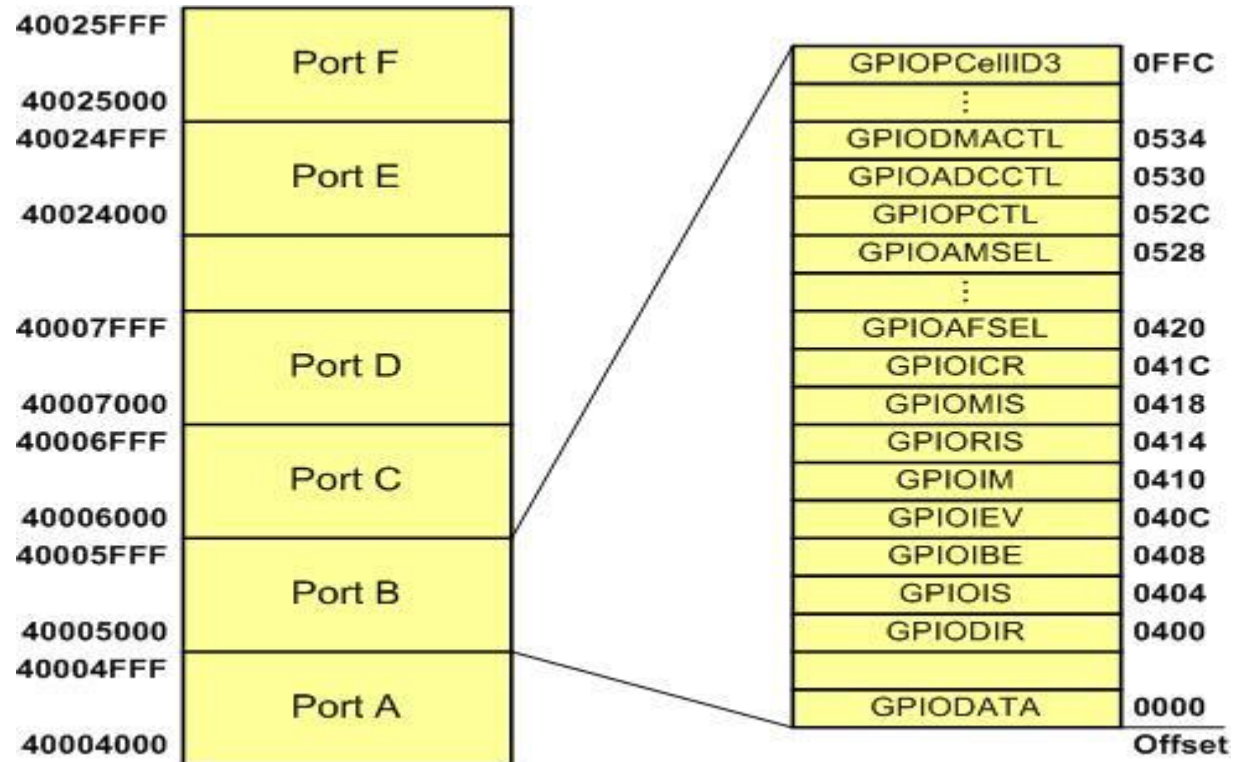
TIVA Launchpad Reference Manual

https://www.ti.com/lit/ug/spmu296/spmu296.pdf?ts=1737133894632&ref_url=https%253A%252F%252Fwww.google.com%252F

Example Project

- In this lecture we will learn how to turn on Red LED on TIVA Launchpad.
- We will first write a code in assembly and then in C.

GPIO Memory Map



Exercise

- Look at the memory map in the previous slide and write down the base address for each port:

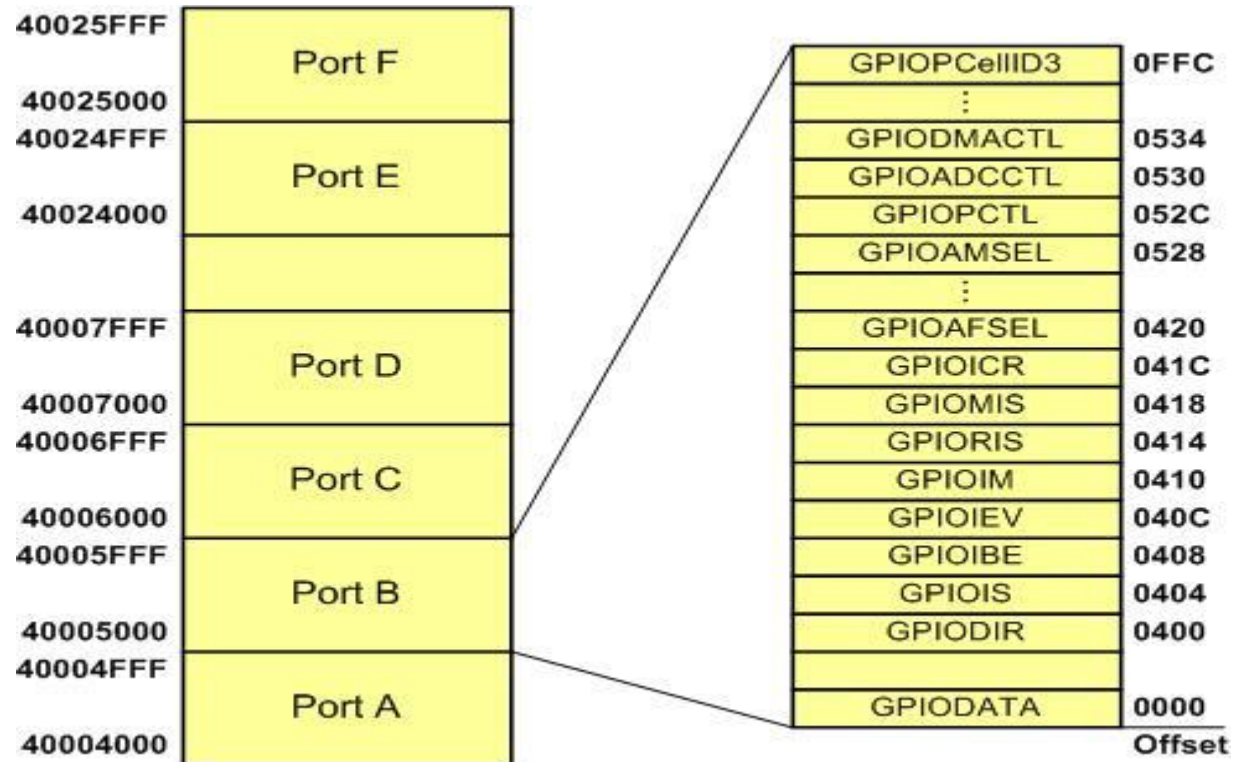
Port	Base Address
A	
B	
C	
D	
E	
F	

Exercise 1

- Look at the memory map in the previous slide and write down the **base address** for each port:

Port	Base Address
A	0x40004000
B	0x40005000
C	0x40006000
D	0x40007000
E	0x40024000
F	0x40025000

GPIO Memory Map



Exercise 2

- Look at the memory map in the previous slide again and write down the address for GPIODATA register for each port:

GPIODATA	Base Address
GPIODATA_A	
GPIODATA_B	
GPIODATA_C	
GPIODATA_D	
GPIODATA_E	
GPIODATA_F	

Exercise 2

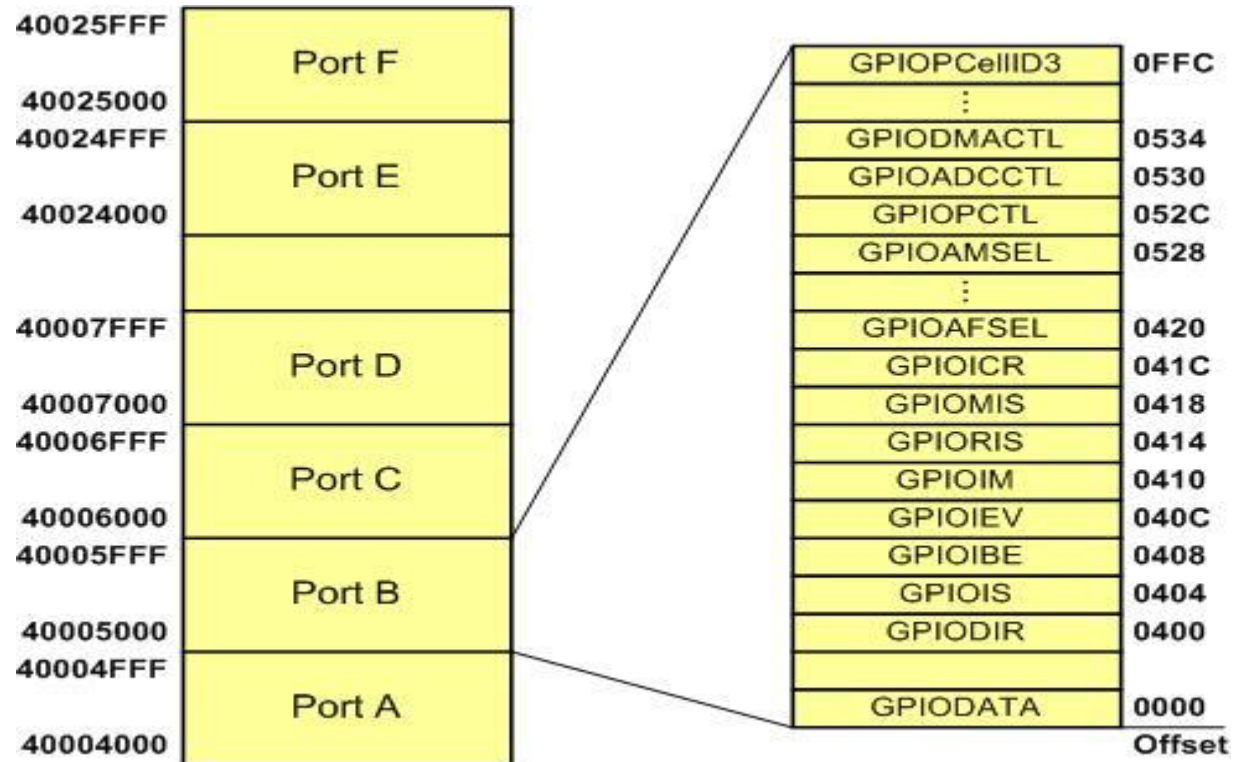
- Look at the memory map in the previous slide again and write down the address for **GPIO DATA register** for each port:

The offset for GPIO DATA is 0000.

$\text{GPIO DATA}_{\text{address}} = \text{Base} + \text{offset}$

GPIO DATA	Base Address
GPIO DATA_A	0x40004000
GPIO DATA_B	0x40005000
GPIO DATA_C	0x40006000
GPIO DATA_D	0x40007000
GPIO DATA_E	0x40024000
GPIO DATA_F	0x40025000

GPIO Memory Map



Exercise 3

- Look at the memory map in the previous slide one more time and write down the address for GPIODIR register for each port:

GPIODIR	Base Address
GPIODIR_A	
GPIODIR_B	
GPIODIR_C	
GPIODIR_D	
GPIODIR_E	
GPIODIR_F	

Exercise 3

- Look at the memory map in the previous slide one more time and write down the address for **GPIODIR register** for each port:

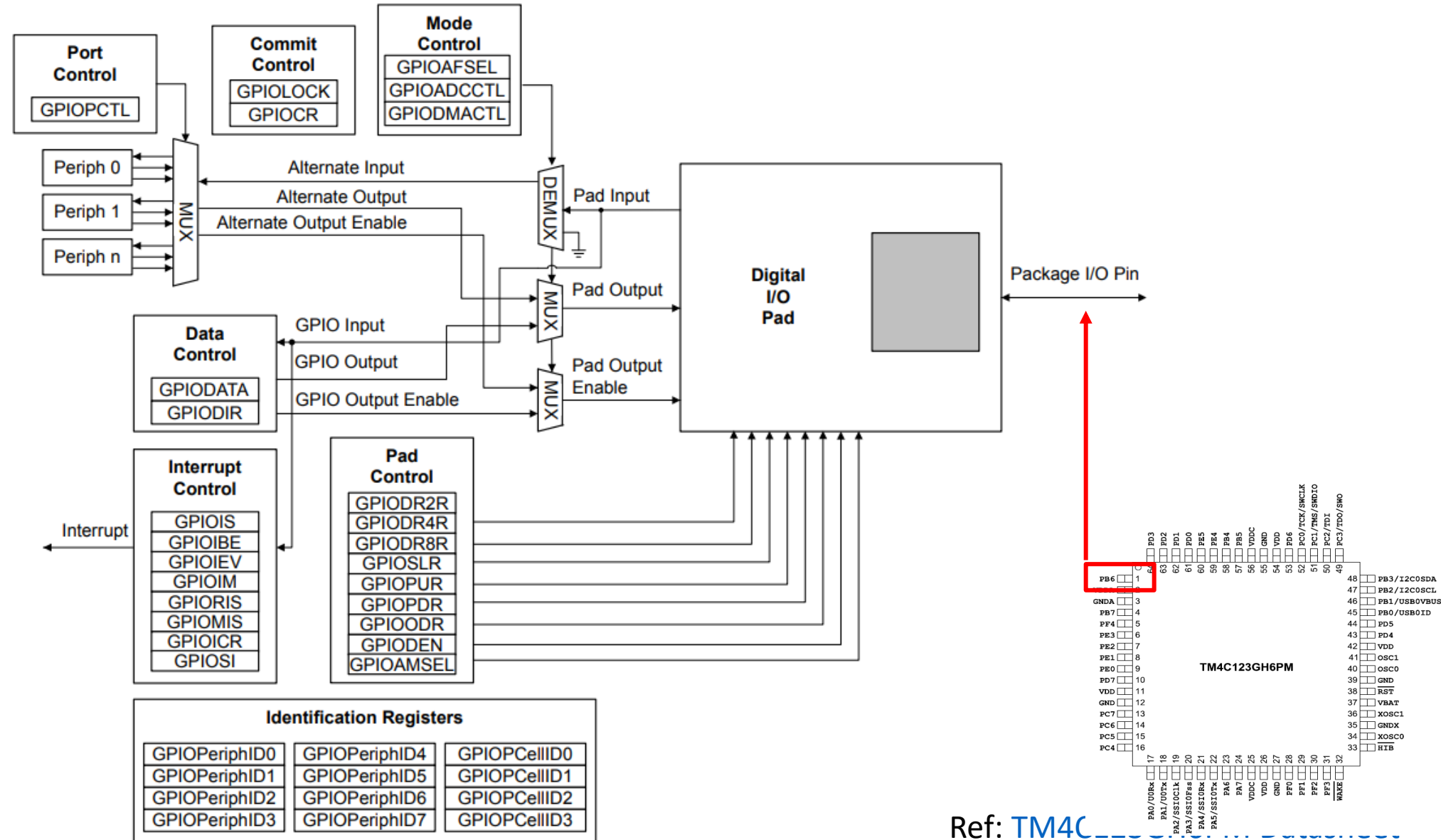
The offset for GPIODIR is 0400.

$\text{GPIODIR}_{\text{address}} = \text{Base} + \text{offset}$

$\text{GPIODIR}_{\text{address}} = \text{Base} + 0x0400$

GPIODIR	Base Address
GPIODIR_A	0x40004400
GPIODIR_B	0x40005400
GPIODIR_C	0x40006400
GPIODIR_D	0x40007400
GPIODIR_E	0x40024400
GPIODIR_F	0x40025400

Figure 10-1. Digital I/O Pads



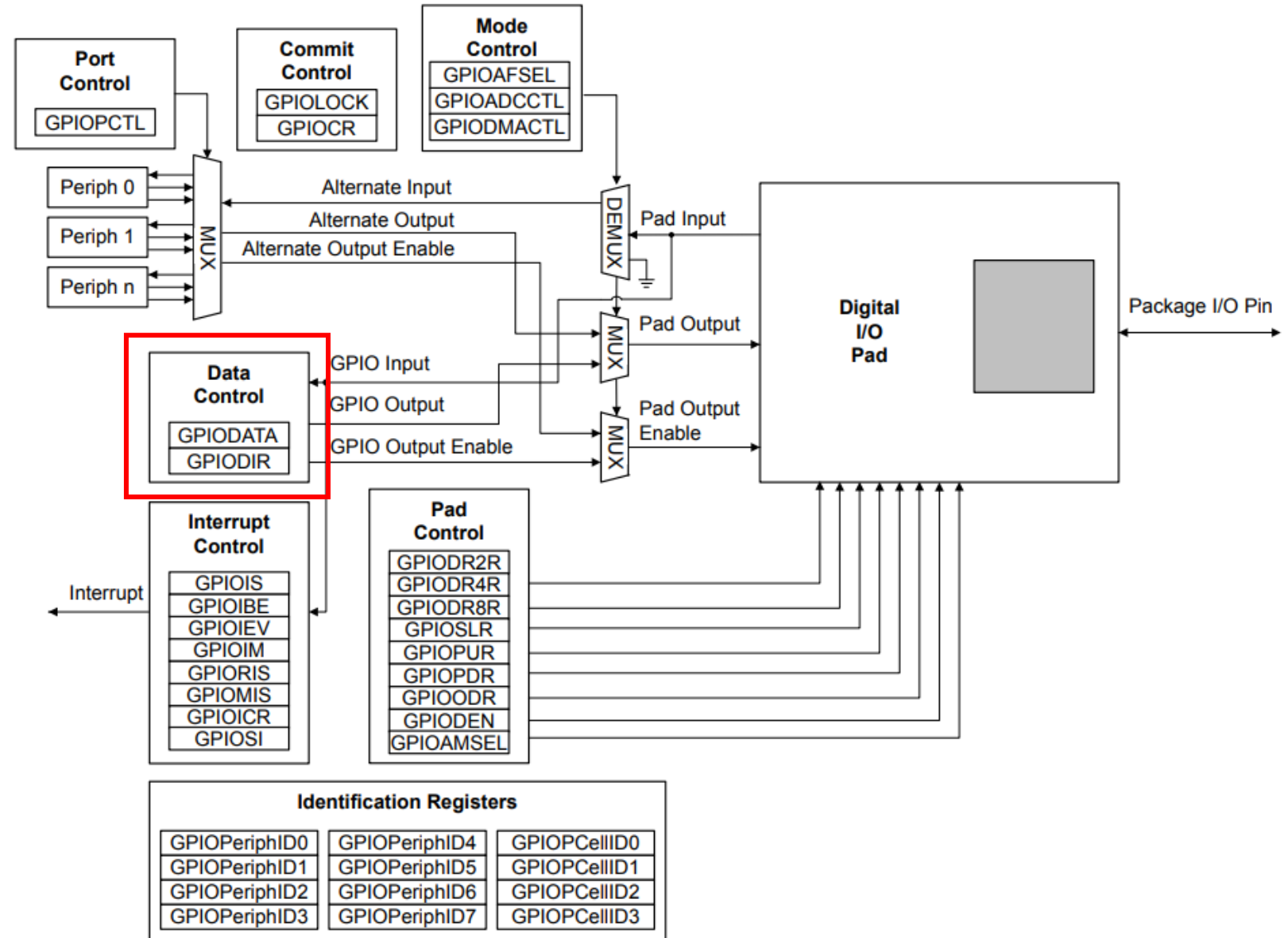
Ref: [TM4C123GH6PM](#)

Data Control

Register:

- GPIODATA
- GPIODIR

Figure 10-1. Digital I/O Pads



Data Register (GPIO DATA)

- The GPIO DATA is located at the offset address of 0x0000 from the base address of port.
- It is a 32 bit register, but we only need the 8 least significant bits.
- Each bit corresponds to one pin on a port.
- For example, in GPIO DATA_A, D0 corresponds to PA0, D1 corresponds to PA1 and etc.



Direction Register (GPIODIR)

- Each GPIO pin can be configured as an **input** or as an **output** pin.
- For example:
 - if we connect a LED to a pin, we need to configure the pin as an output.
 - If we connect a switch to a pin, we need to configure the pin as an input.
- GPIODIR register is used for this purpose.



Note: D0 to D7 are used to set the direction for pins 0 to 7 of the port.

1: Output

0: Input

Direction Register (GPIODIR)

- The address of GPIO Direction Register is located at the offset address of 0x400 from the base address of that port.
- For configuration, we need to write 0 to configure the pin as an input and we need to write 1 to configure the pin as an output.

Input: GPIODIR = 0

Output: GPIODIR = 1



Note: D0 to D7 are used to set the direction for pins 0 to 7 of the port.

1: Output

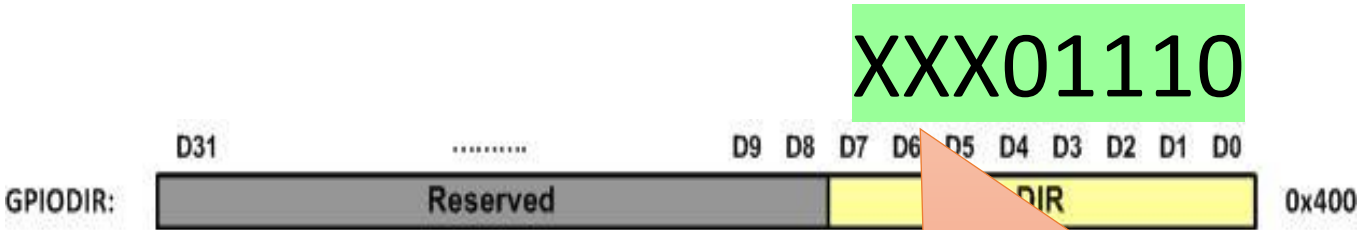
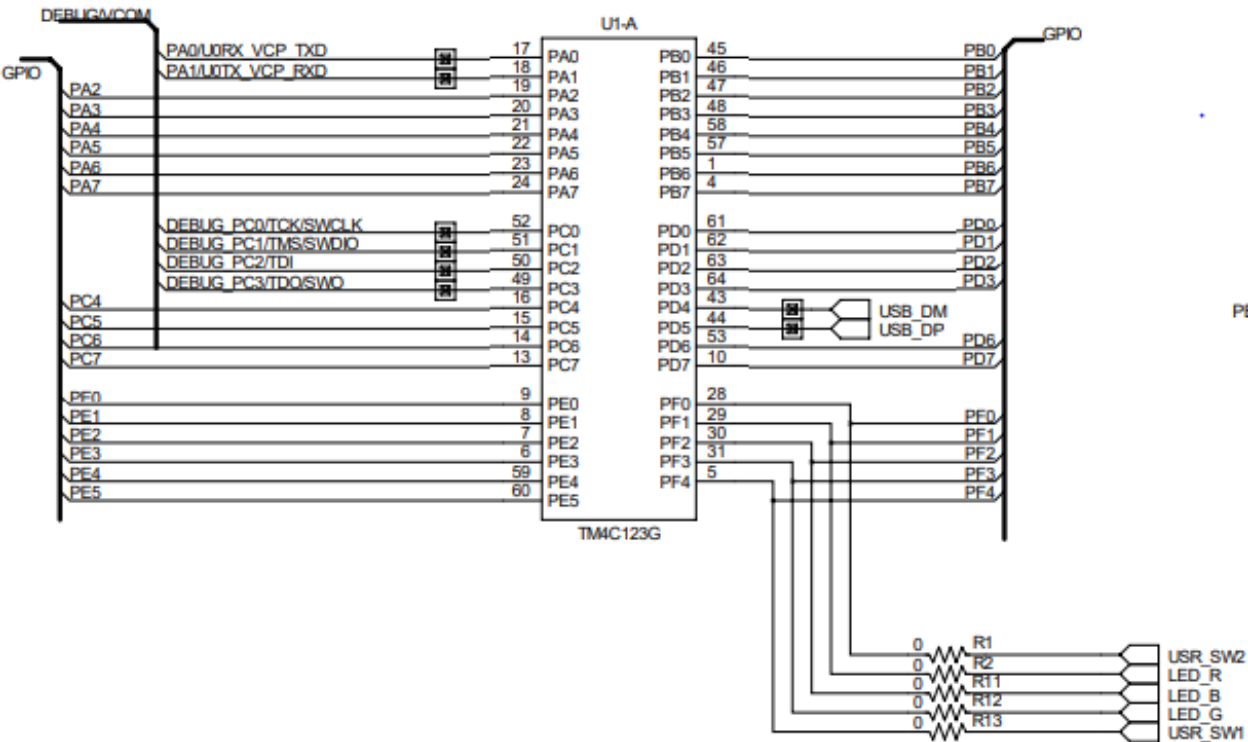
0: Input

Data Direction Operation

- The **GPIO Direction (GPIODIR) register** is used to configure each individual pin as an input or output.
- When the **data direction bit is cleared**, the GPIO is configured as an **input**, and the corresponding data register bit captures and stores the value on the GPIO port.
- When **the data direction bit is set**, the GPIO is configured as an **output**, and the corresponding data register bit is driven out on the GPIO port.

Example

- In this example, PF0 and PF4 are configured as inputs and PF1, PF2, PF3 are configured as outputs.
- Therefore, the value that we will write to GPIODIR_F is:



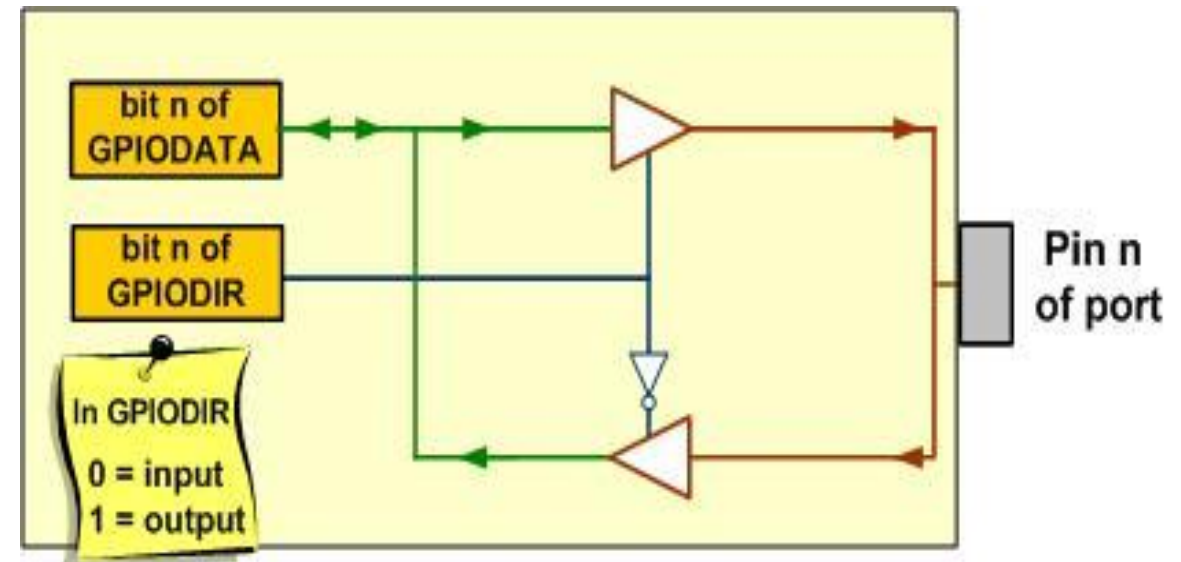
X means that these bits are unchanged.

Device	GPIO
Switch 2	GPIOF Pin 0
Red LED	GPIOF Pin 1
Blue LED	GPIOF Pin 2
Green LED	GPIOF Pin 3
Switch 1	GPIOF Pin 4

Direction and Data Register

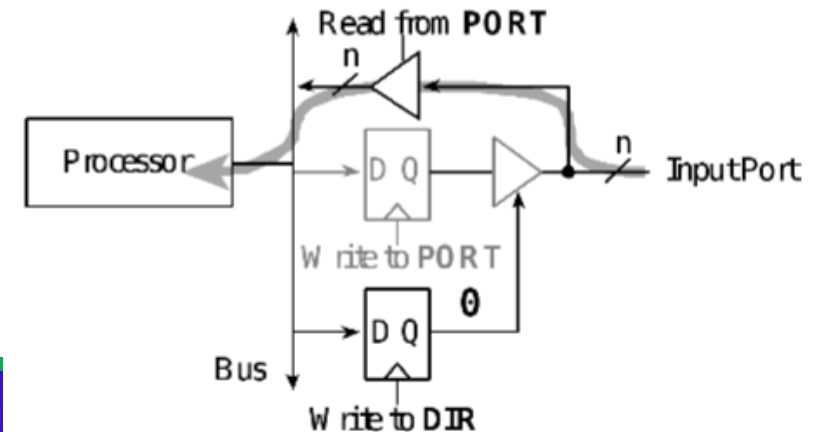
Generally, every microcontroller has minimum of two registers associated with each I/O port:

- Data Register (GPIODATA)
- Direction Register (GPIODIR)



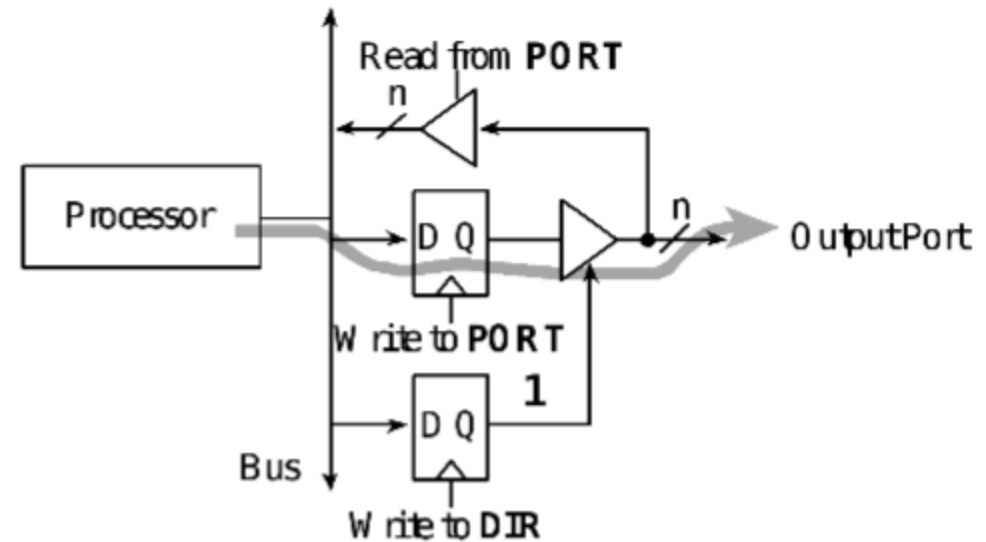
Input Port

- An **input port** allows the software to read external digital signals. That means a read cycle access from the port address returns the value existing on the input at that time.
- To make a pin input, we write **0** to its **direction register**.
- TM4C microcontrollers are 5V-tolerant:
 - Input voltage between **2.145V** and **5V** is considered **HIGH**.
 - Input voltage between **0** and **1.155V** is considered **LOW**.



Output Port

- A readable **output port** allows the software to generate external digital signals.
- To make a pin input, we write **1** to its **direction register**.



GPIO Digital Enable Register (GPIODEN)

- Each GPIO pin can be configured as analog or digital pin.
- The GPIODEN (Digital Enable) special function register is used to **enable digital function** on an I/O pin instead of analog function.

Digital Function: GPIODEN = 1

Analog Function: GPIODEN = 0



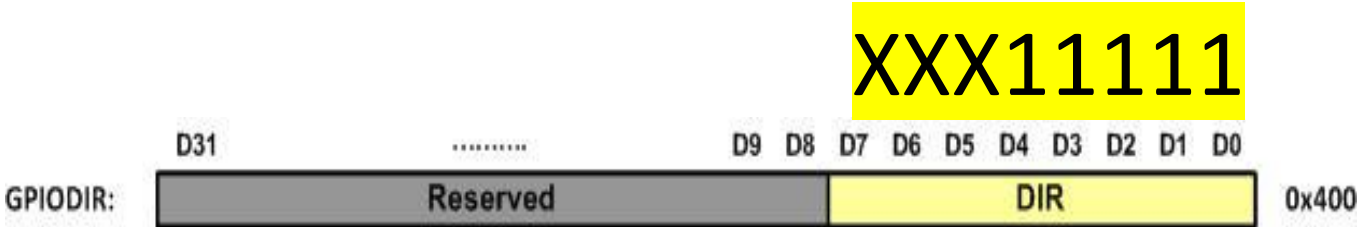
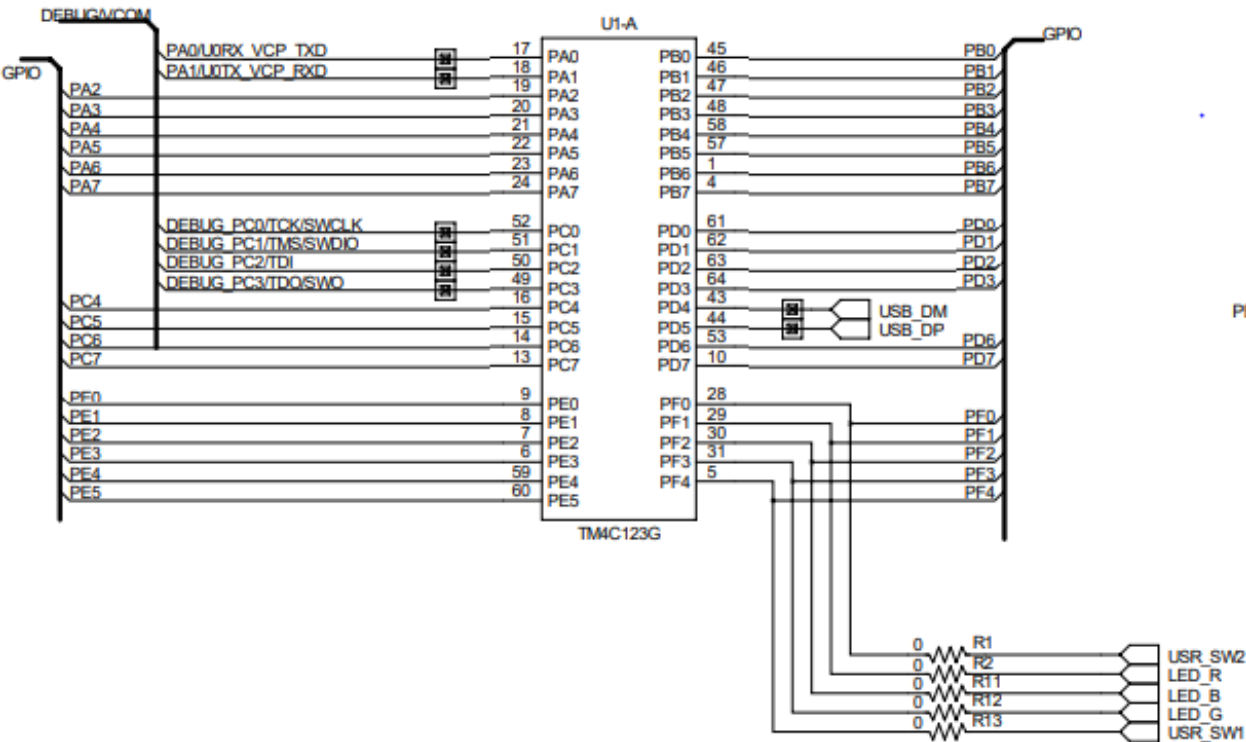
Note: D0 to D7 are used to Enable the digital circuit for pins 0 to 7 of the port.

1: The digital functions for the corresponding pin are enabled.

0: The digital functions for the corresponding pin are disabled.

Example

- In this example, PF0 to PF4 are configured as digital pin.
- Therefore, the value that we will write to GPIODEN_F is:

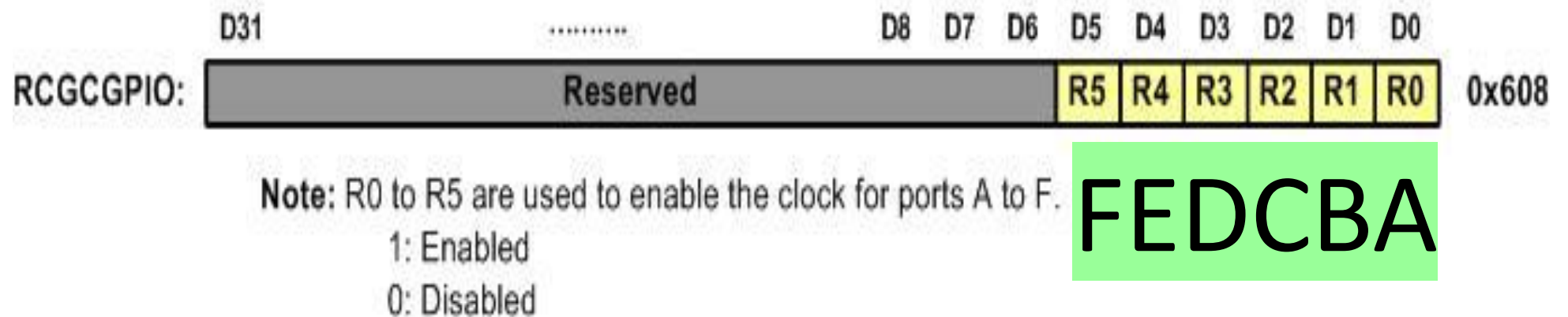


Device	GPIO
Switch 2	GPIOF Pin 0
Red LED	GPIOF Pin 1
Blue LED	GPIOF Pin 2
Green LED	GPIOF Pin 3
Switch 1	GPIOF Pin 4

GPIO Clock Enable (RCGCGPIO) (for all the IO Ports)

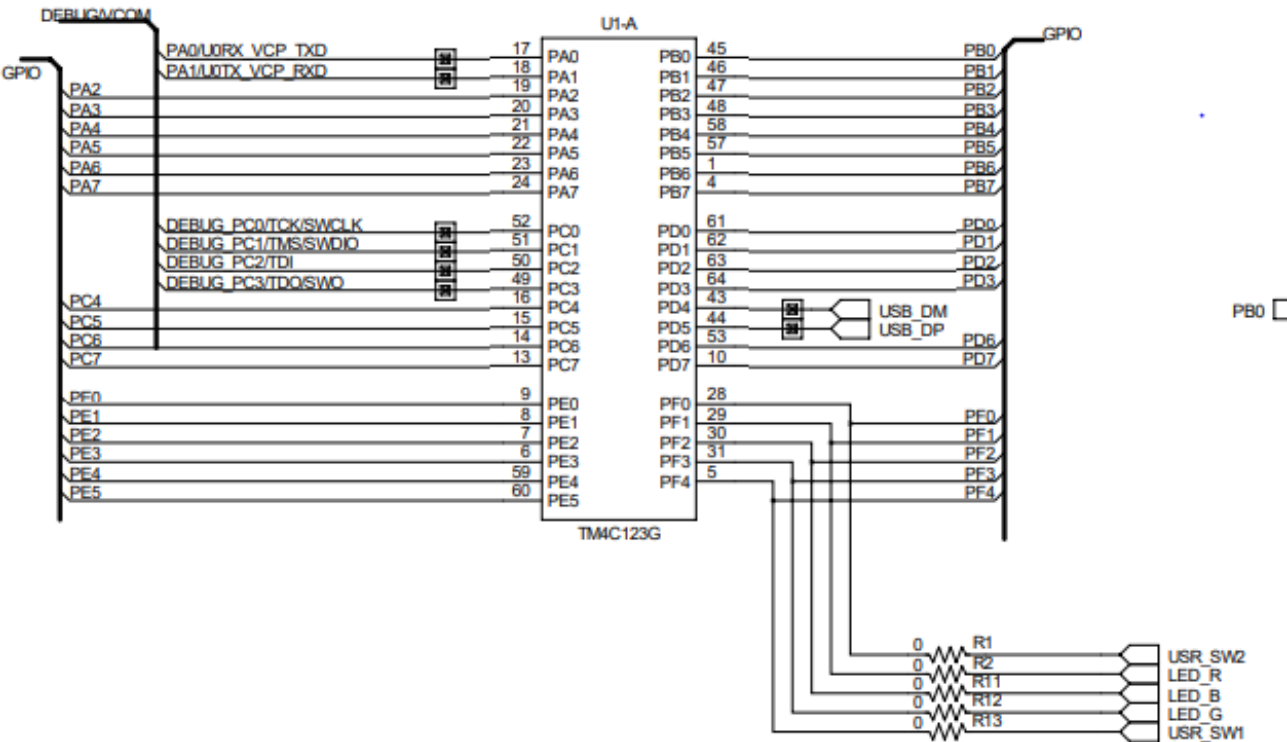
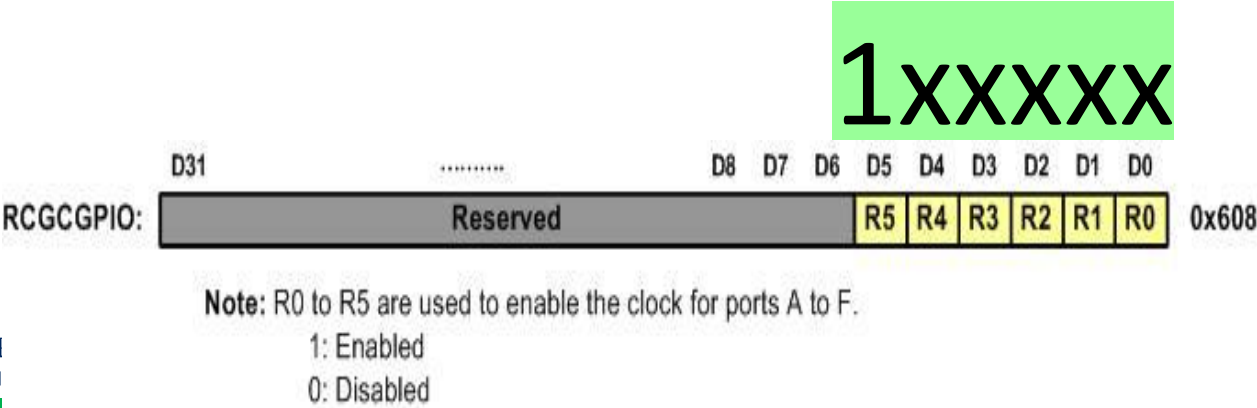
The RCGCGPIO is used to **enable the clock source** for the I/O Circuitry.

There is only one RCGCGPIO special function register for **all the ports** and each bit of that register is used to enable the clock source to one of the ports.



Example

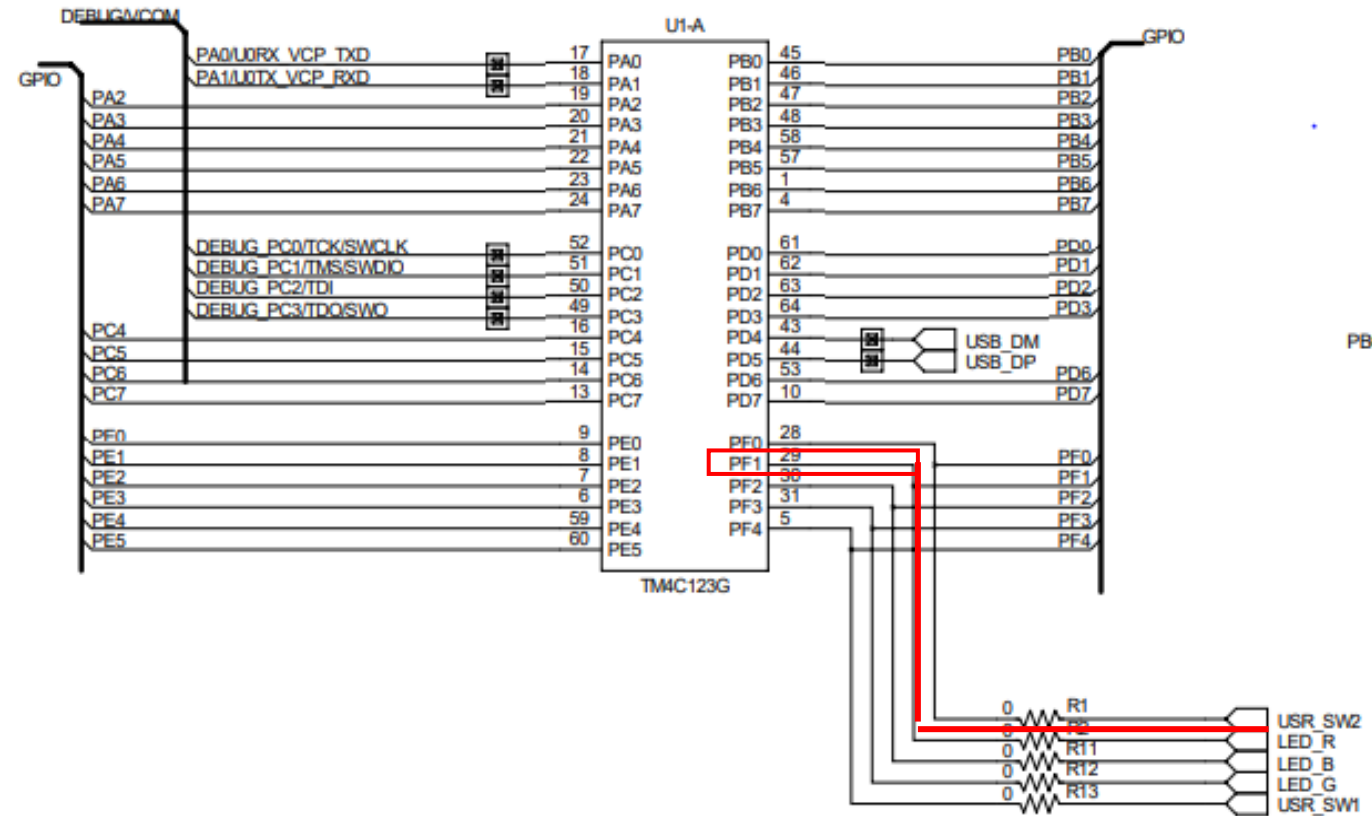
- In this example, we are using port F.
- Therefore, we need to make sure that the clock is enabled for Port F. The value that we will write to RCGCGPIO is:



Device	GPIO
Switch 2	GPIOF Pin 0
Red LED	GPIOF Pin 1
Blue LED	GPIOF Pin 2
Green LED	GPIOF Pin 3
Switch 1	GPIOF Pin 4

Example Project

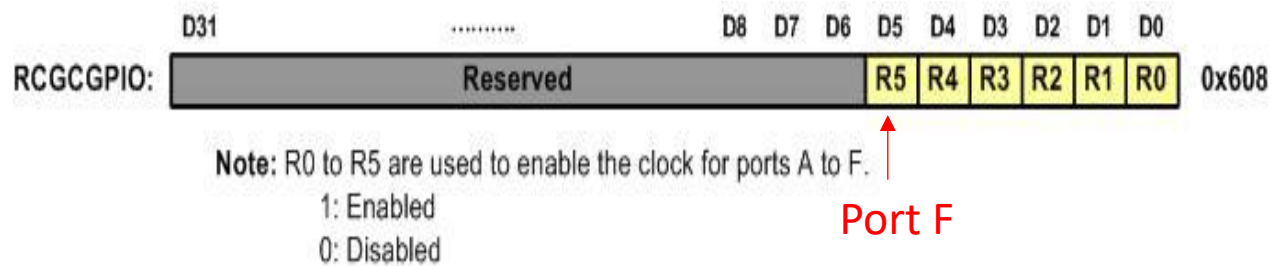
- In this lecture we will learn how to turn on Red LED on TIVA Launchpad, which is connected to PF1.
- We will first write a code in assembly and then in C.



Steps to Configure a GPIO Pin as Output:

1. **Enable Clock:** Set the corresponding bit in the **RCGCGPIO** register to enable the clock for the GPIO port.
2. **Enable Digital Function:** Set the corresponding bit in the **GPIODEN** register to enable the pin as a digital function.
3. **Configure as Output:** Set the corresponding bit in the **GPIODIR** register to configure the pin as an output.
4. **Set Output Value:** Write the desired output value to the **GPIODATA** register.

Step 1: Enable Clock for Port F



Reset Value: 0x00000000

0b0000,0000,0000,0000,0000,0000,0000,0000

Set Port F: 0b0000,0000,0000,0000,0000,0000,0010,0000
0x00000020

Port F

Assembly Logical Operation

ORR

In1	In2	OUT
0	0	0
0	1	1
1	0	1
1	1	1

AND

In1	In2	OUT
0	0	0
0	1	0
1	0	0
1	1	1

EOR

In1	In2	OUT
0	0	0
0	1	1
1	0	1
1	1	0

ORR

ORR R0, R0, #0x20 (Note: 0x20 = 0x00000020)

Reset_value = Reset_value **ORR** 0x20

ORR

0b0000,0000,0000,0000,0000,0000,0000,0000

0b0000,0000,0000,0000,0000,0000,00**1**0,0000

0b0000,0000,0000,0000,0000,0000,00**1**0,0000 = 0x00000020

RCGCGPIO Memory Address

- RCGCGPIO is one of the **System Control** registers.
- The base address of System Control is 0x400FE000.
- The RCGCGPIO is at 0x608 offset from the System Control Base.
- $\text{RCGCGPIO}_{\text{address}} = \text{Base}_{\text{address}} + \text{offset}$
- $\text{RCGCGPIO}_{\text{address}} = 0x400FE000 + 0x608 = 0x400FE608$
- How to implement it in assembly:

```
RCGCGPIO EQU 0x400FE608
```

Step 1: Enable Clock for Port F

```
; load R1 with address of RCGCGPIO register
LDR R1, =RCGCGPIO

; load the value of R2 with the content of RCGCGPIO register
LDR R2, [R1]

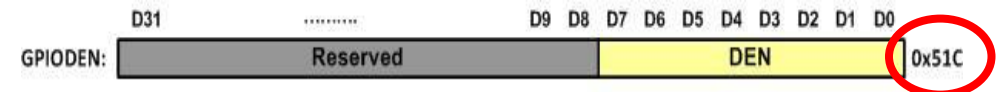
; ORR R2 with 0x20 to set the bit for Port F and enable the clock
ORR R2, 0x20

; store the new value in RCGCGPIO register
STR R2, [R1]
```

Step 2: Enable Digital Function

- First, let's find the address of GPIODEN for Port F
- The base address of Port F = **0x40025000**
- The offset of GPIODEN = **0x51C**
- The address of GPIODEN_F = **0x40025000** + **0x51C** = **0x4002551C**
- How to implement it in assembly:

```
GPIODEN_F EQU 0x4002551C
```



Note: D0 to D7 are used to Enable the digital circuit for pins 0 to 7 of the port.

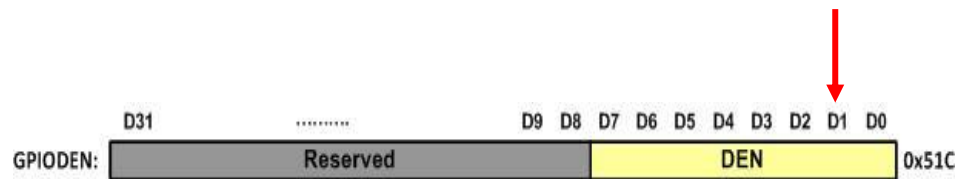
1: The digital functions for the corresponding pin are enabled.

0: The digital functions for the corresponding pin are disabled.

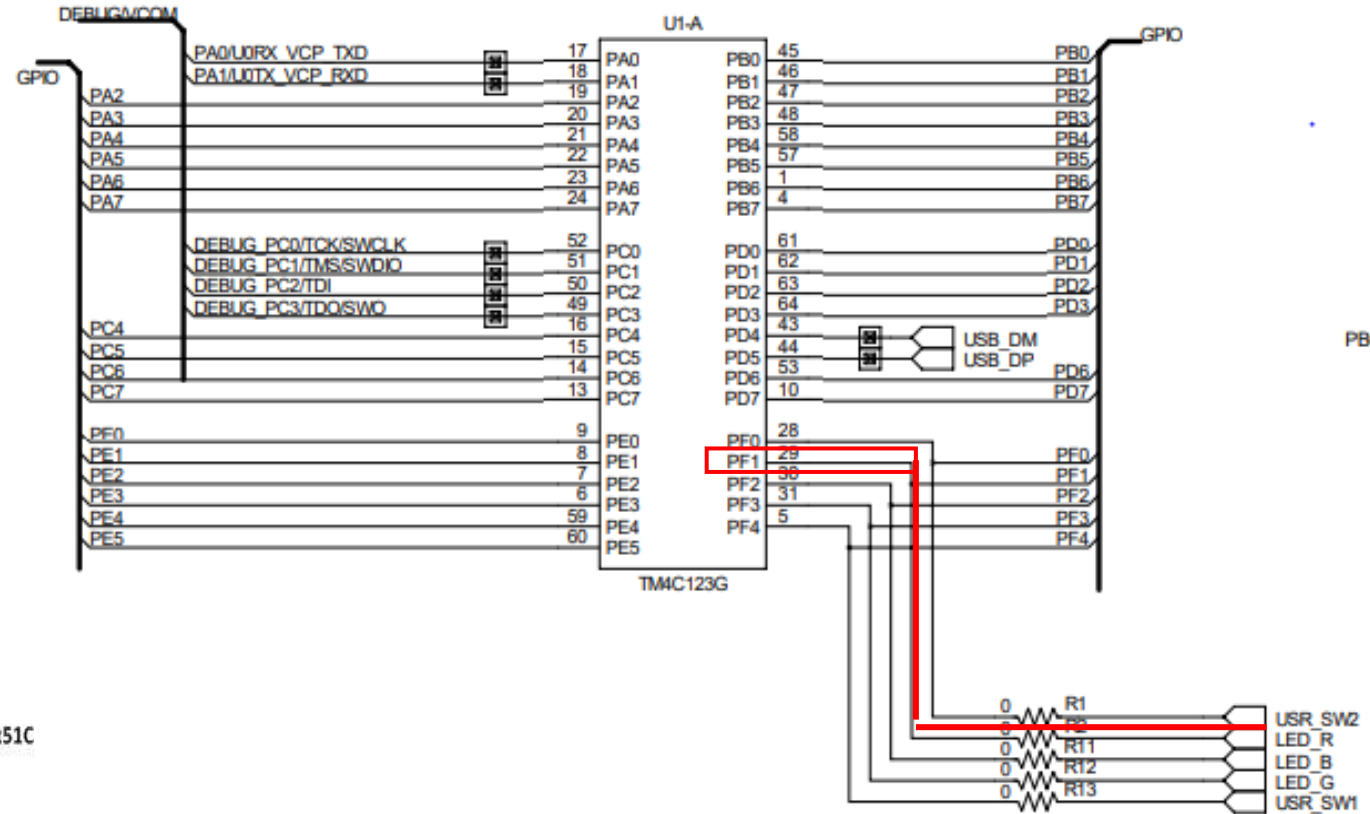
Step 2: Enable Digital Function (cntd)

- **Red LED** is connected to **PF1**.
- Therefore:
PF1 is enabled for digital function.

D1 = 1



Note: D0 to D7 are used to Enable the digital circuit for pins 0 to 7 of the port.
1: The digital functions for the corresponding pin are enabled.
0: The digital functions for the corresponding pin are disabled.



Step 2: Enable Digital Function (cntd)

GPIODEN is a 32 bit register. However, we want to only change D1 in this register. we do not want to change any other bit in this register.

Reset Value: 0x00000000

0b0000,0000,0000,0000,0000,0000,0000,0000

Set Bit D1: 0b0000,0000,0000,0000,0000,0000,0000,00**1**0

0x0000000**2**

Step 2: Enable Digital Function (cntd)

```
; load R3 with address of GPIODEN_F register
```

```
LDR R3, =GPIODEN_F
```

```
; load the value of R4 with the content of GPIODEN_F register
```

```
LDR R4, [R3]
```

```
; ORR R4 with 0x02 to set the bit D1
```

```
ORR R4, 0x02
```

```
; store the new value in GPIODEN_F register
```

```
STR R4, [R3]
```

Step 4: Configure PF1 as Output

- The address of GPIODATA_F = 0x40025000 (refer to Slide 29)
- We declare a label in assembly:

```
GPIODIR_F EQU 0x40025400
```

Step 3: Configure PF1 as Output (contd)

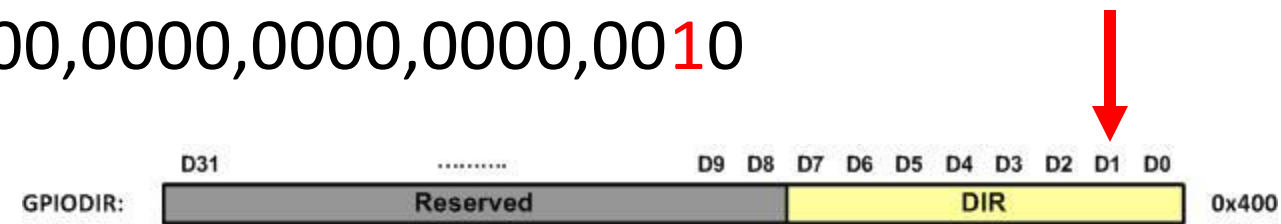
GPIO_DIR is a 32 bit register. However, we want to only change D1 in this register. we do not want to change any other bit in this register.

Reset Value: 0x00000000

0b0000,0000,0000,0000,0000,0000,0000,0000

Set Bit D1: 0b0000,0000,0000,0000,0000,0000,0000,00**1**0

0x00000000**2**



Note: D0 to D7 are used to set the direction for pins 0 to 7 of the port.

1: Output

0: Input

Step 3: Configure PF1 as Output (contd)

```
; load R5 with address of GPIODIR_F register
LDR R5, =GPIODIR_F

; load the value of R6 with the content of GPIODIR_F register
LDR R6, [R5]

; ORR R6 with 0x02 to set the bit D1
ORR R6, 0x02

; store the new value in GPIODIR_F register
STR R6, [R5]
```

Step 4: Write to Data register

- The address of GPIODIR_F = 0x40025400 (refer to Slide 26)
- We declare a label in assembly:

```
GPIODATA_F EQU 0x40025000
```

Step 3: Write to Data register (contd)

GPIO DATA is a 32 bit register. However, we want to only write 1 to D1 (PF1) in this register to turn on the LED. we do not want to change any other bit in this register.

Reset Value: 0x00000000

0b0000,0000,0000,0000,0000,0000,0000,0000

Set Bit D1: 0b0000,0000,0000,0000,0000,0000,0000,00**1**0

0x00000000**2**



Step 3: Configure PF1 as Output (contd)

```
; load R5 with address of GPIODIR_F register
```

```
LDR R5, =GPIODIR_F
```

```
; load the value of R6 with the content of GPIODIR_F register
```

```
LDR R6, [R5]
```

```
; ORR R6 with 0x02 to set the bit D1
```

```
ORR R6, 0x02
```

```
; store the new value in GPIODIR_F register
```

```
STR R6, [R5]
```

```

1  /* p2.4.c: Toggling a single LED
2
3  /* This program turns on the red LED and toggles the blue LED 0.5 sec on and 0.5 sec off. */
4
5  #include "TM4C123GH6PM.h"
6
7  void delayMs(int n);
8
9  int main(void)
10 {
11
12     /* enable clock to GPIOF at clock gating control register */
13     SYSCCTL->RCGCGPIO |= 0x20;
14     /* enable the GPIO pins for the LED (PF3, 2 1) as output */
15     GPIOF->DIR = 0x0E;
16     /* enable the GPIO pins for digital function */
17     GPIOF->DEN = 0x0E;
18     /* turn on red LED only and leave it on */
19     GPIOF->DATA = 0x02;
20
21     while(1)
22     {
23         GPIOF->DATA |= 4;    /* turn on blue LED */
24         delayMs(500);
25
26         GPIOF->DATA &= ~4;   /* turn off blue LED */
27         delayMs(500);
28     }
29 }
30

```

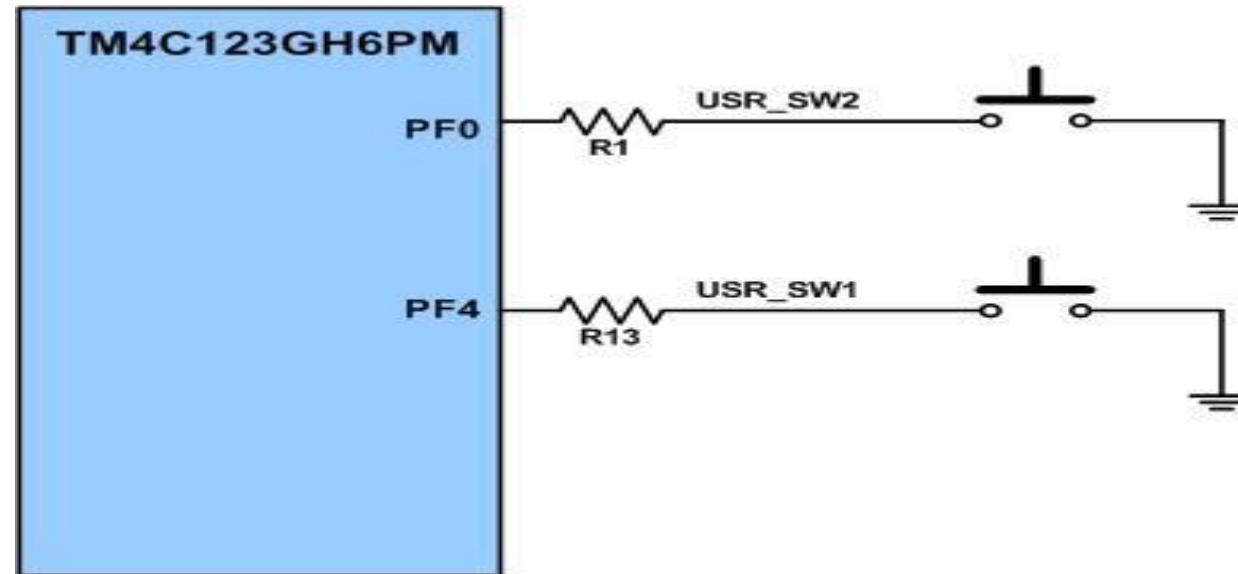
```

1  /* p2_6.c: Toggling all bits of Port A at 16 Hz (25 ms on, 25 ms off) */
2
3
4  #include "TM4C123GH6PM.h"
5
6  void delayMs(int n);
7
8  int main(void)
9  {
10     /* enable clock to GPIOA at clock gating control register */
11     SYSCTL->RCGCGPIO |= 1;
12     /* enable the GPIOA pins as output */
13     GPIOA->DIR = 0xFF;
14     /* enable the GPIOA pins for digital function */
15     GPIOA->DEN = 0xFF;
16
17     while(1)
18     {
19         GPIOA->DATA = 0xFF; /* turn on all the pins of Port A */
20         delayMs(25);
21
22         GPIOA->DATA = 0;    /* turn off all the pins of Port A */
23         delayMs(25);
24     }
25 }
26
27 /* delay n milliseconds (16 MHz CPU clock) */
28 void delayMs(int n)
29 {
30     int i, j;
31     for(i = 0 ; i < n; i++)
32         for(j = 0; j < 3180; j++)
33             {} /* do nothing for 1 ms */
34 }

```

Reading a switch in TI Tiva LaunchPad

(Push-button Switches Connected to the Microcontroller in the Tiva LaunchPad Board)



```

1  /* p2_7.c: Read a switch and write it to the LED */
2
3  /* This program reads SW1 of Tiva LaunchPad and writes the inverse of
4  the value to the green LED. SW1 is low when pressed (Normally High). LED is on when high. */
5
6
7  #include "TM4C123GH6PM.h"
8
9  int main(void)
10 {
11     unsigned int value;
12     SYSCTL->RCGCGPIO |= 0x20;    /* enable clock to GPIOF */
13     GPIOF->DIR = 0x08;          /* set PORTF3 pin as output (LED) pin */
14                                /* and PORTF4 as input, SW1 is on PORTF4 */
15     GPIOF->DEN = 0x18;          /* set PORTF pins 4-3 as digital pins */
16     GPIOF->PUR = 0x10;          /* enable pull up for pin 4 */
17
18     while(1)
19     {
20         value = GPIOF->DATA;    /* read data from PORTF */
21         value = ~value;        /* switch is low active; LED is high active */
22         value = value >> 1;    /* shift it right to display on green LED */
23         GPIOF->DATA = value;    /* put it on the green LED */
24     }
25 }
26

```

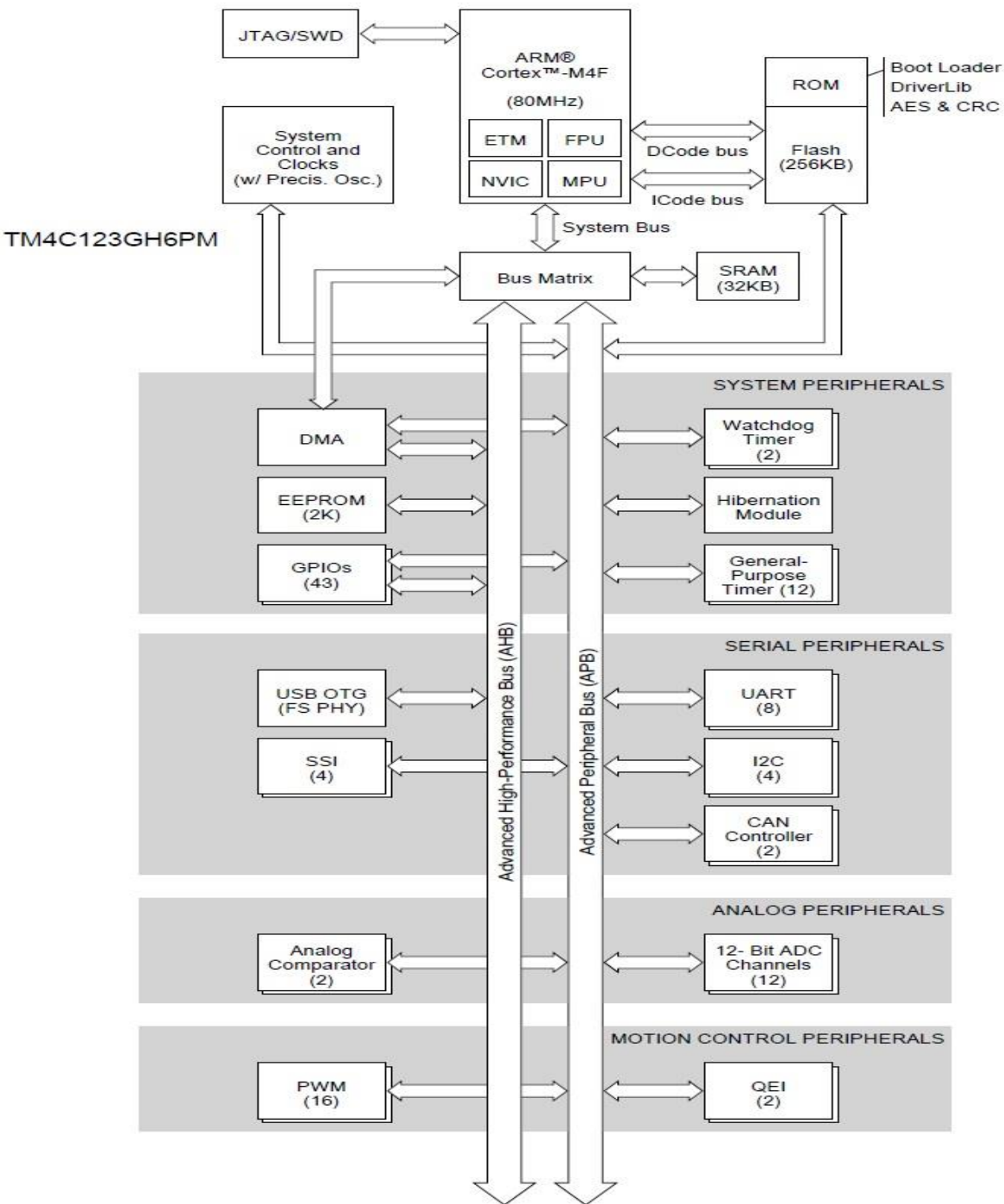
```

3  /* This program reads SW2 of Tiva LaunchPad and write the inverse of the value to the red LED.
4  SW2 is low when pressed. LED is on when high. */
5
6  /* SW2 is connected to PORTF0, which is an NMI pin. */
7  /* In order to use this pin for any function other than NMI, the pin needs be unlocked first. */
8
9
10 #include "TM4C123GH6PM.h"
11
12 int main(void)
13 {
14     unsigned int value;
15     SYSCTL->RCGCGPIO |= 0x20;    /* enable clock to GPIOF */
16
17     GPIOF->LOCK = 0x4C4F434B;    /* unlock commit register */
18     GPIOF->CR = 0x01;            /* make PORTF0 configurable */
19
20     GPIOF->DIR = 0x02;            /* set PORTF1 pin as output (LED) pin */
21                                 /* and PORTF0 as input, SW2 is on PORTF0 */
22     GPIOF->DEN = 0x03;            /* set PORTF pins 1-0 as digital pins */
23     GPIOF->PUR = 0x01;            /* enable pull up for pin 0 */
24
25     while(1)
26     {
27         value = GPIOF->DATA;      /* read data from PORTF */
28         value = ~value;           /* switch is low active; LED is high active */
29         value = value << 1;       /* shift it left to display on red LED */
30         GPIOF->DATA = value;      /* put it on red LED */
31     }
32 }
33

```

TI Tiva TM4C123GH6PM Microcontroller

High-Level Block Diagram



APB vs. AHB

APB (Advanced Peripheral Bus)

AHP (Advanced High Performance Peripheral Bus)

- AHP is much faster than APB.
- AHP needs 1 clock cycle to access peripherals.
- ABP needs at least 2 clock cycle to access peripherals.
- The base addresses are different for AHP and ABP.

Bi-directional GPIO Pins

All GPIO pins are bidirectional, which means that they can be individually configured in the code to work as input or output.

Therefore, each port can connect to input devices, output devices, or mix of both.

Reference

1. Mazidi, M., Chen, S., Naimi, S., Naimi, S., Chen, S. (2016). TI TIVA ARM Programming For Embedded Systems, 1st Edition. MicroDigitalEd. ISBN-13: 978-0997925920
2. Valvano, J., (2019). Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, 5th edition, 6th printing